

Project Report:

Mini-Project

จัดทำโดย

- | | | |
|-----------------|-------------|------------|
| 1) นายชัตติยรัฐ | พิมาทัย | (68010107) |
| 2) นายจิราวิท | สุขโขทัย | (68010166) |
| 3) ธีรวัสส์ | กระกุลขยัน | (68010406) |
| 4) นายนนรพัชร์ | น้อยสวัสดิ์ | (68010558) |

บทที่ 1

บทนำ

บทนำ: ภาพรวมของระบบจัดการรถไฟ (Railway Management System)

ระบบจัดการรถไฟ (Railway Management System) เป็นระบบที่ถูกออกแบบและพัฒนาขึ้นมาเพื่ออำนวยความสะดวกในการบริหารจัดการการเดินทางรถไฟแบบครบวงจร ระบบถูกสร้างขึ้นด้วยสถาปัตยกรรมการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming - OOP) ที่มีความยืดหยุ่นสูง โดยครอบคลุมตั้งแต่การจัดการโครงสร้างพื้นฐานของเครือข่ายรถไฟ ไปจนถึงการให้บริการแก่ผู้โดยสาร เพื่อให้การจองตั๋วและการควบคุมการเดินทางเป็นไปอย่างมีประสิทธิภาพ ถูกต้อง และตรวจสอบได้

ระบบนี้ถูกออกแบบมาเพื่อรองรับผู้ใช้งาน 3 กลุ่มหลัก (Actors) ได้แก่:

1. **ผู้โดยสาร (Customer / User):** ผู้ใช้งานที่สามารถเข้ามาดูข้อมูลเส้นทางรอบรถ, สมัครสมาชิก, ทำการจองที่นั่ง, ชำระเงิน, และจัดการตั๋วโดยสารของตนเองได้
2. **เจ้าหน้าที่ประจำสถานี (Station Officer):** บุคลากรที่มีหน้าที่ตรวจสอบความถูกต้องของตั๋วโดยสาร (Verify Ticket) และควบคุมสถานะการเข้า-ออกสถานีของขบวนรถ (Trip Enter/Exit)
3. **ผู้ดูแลระบบ (System Administrator):** ผู้ที่มีอำนาจในการจัดการโครงสร้างหลักของระบบ เช่น การเพิ่ม/ลดเส้นทาง (Route), ขบวนรถ (Train), จัดการตารางเดินรถ (Schedule), จัดการรอบรถ (Trip) และปรับเปลี่ยนค่าธรรมเนียมต่างๆ

โมดูลและฟังก์ชันการทำงานหลักของระบบ (Core Modules)

ระบบถูกแบ่งสัดส่วนการทำงานออกเป็นโมดูลหลักๆ เพื่อให้ง่ายต่อการจัดการ ดังนี้:

- **1. ระบบจัดการเส้นทางและตารางเวลา (Route & Schedule Management)** รองรับการสร้างเส้นทางเดินรถ (Route) ที่ประกอบไปด้วยสถานีย่อยต่างๆ (Station/Stop) และมีการกำหนดตารางเวลามาตรฐาน (Schedule) สำหรับแต่ละเส้นทาง ทำให้ระบบสามารถสร้างรอบการเดินทางจริง (Trip) ในแต่ละวันได้อย่างแม่นยำ
- **2. ระบบบริหารจัดการขบวนรถและที่นั่ง (Train & Carriage Management)** รองรับรถไฟหลากหลายประเภท (เช่น Commuter, Speeder, Splinter) โดยแต่ละขบวนสามารถประกอบด้วยตู้โดยสาร (Carriage) ที่แบ่งระดับชั้นได้ เช่น ชั้นธรรมดา (Normal) และชั้นธุรกิจ (Business) ซึ่งระบบสามารถคำนวณและติดตามสถานะของที่นั่ง (Seat Status) ว่าว่างหรือถูกจองไปแล้วได้อย่างเรียลไทม์ ป้องกันปัญหาการจองที่นั่งซ้ำซ้อน
- **3. ระบบการจองและชำระเงิน (Booking & Payment System)** ผู้โดยสารสามารถทำการจองตั๋ว (Booking) โดยระบุสถานีต้นทาง-ปลายทาง ขบวนรถ และที่นั่งที่ต้องการได้ ระบบรองรับช่องทางการชำระเงินที่หลากหลาย ทั้ง E-Wallet, บัตรเครดิต/เดบิต, การโอนเงินผ่านธนาคาร และ Direct Debit เมื่อชำระเงินสำเร็จ ระบบจะออกตั๋วโดยสาร (Ticket) ให้โดยอัตโนมัติ
- **4. ระบบสมาชิกและสิทธิประโยชน์ (Customer & Reward System)** ผู้โดยสารที่ลงทะเบียนสามารถสะสมคะแนน (Reward Points) จากการเดินทาง และระบบยังรองรับประเภทลูกค้าที่แตกต่างกัน เช่น สมาชิกทั่วไป (Member) หรือผู้สูงอายุ (Senior) ซึ่งอาจมีสิทธิประโยชน์หรือส่วนลดที่แตกต่างกันในอนาคต
- **5. ระบบควบคุมและประวัติการใช้งาน (Tracking & History)** ระบบมีการเก็บสถานะอย่างรัดกุมด้วยระบบ Enumeration (เช่น สถานะตัว, สถานะการจอง, สถานะเส้นทาง) รวมถึงมีการเก็บบันทึกประวัติการทำรายการของพนักงาน (Usage History) และระบบ Transaction ที่รองรับการ Rollback เพื่อป้องกันข้อผิดพลาดของข้อมูล

สรุป

ระบบจัดการรถไฟฟ้า เป็นระบบที่นำเทคโนโลยีมาช่วยเชื่อมโยงระหว่างผู้ให้บริการและผู้โดยสารอย่างไร้รอยต่อ (Seamless Experience) ผ่านโครงสร้างโค้ดที่ได้มาตรฐาน (ทำงานร่วมกับเทคโนโลยี MCP - Model Context Protocol) เพื่อให้การบริหารจัดการเดินรถไฟฟ้าในแต่ละวันเป็นไปอย่างราบรื่น ปลอดภัย และตอบสนองต่อความต้องการของผู้ใช้งานในยุคดิจิทัลได้อย่างแท้จริง

บทที่ 2 Use Case Diagram



Use Case Name	View Route
Actor	USER
Descriptions	แสดงเส้นทางรถไฟทั้งหมดที่เปิดให้บริการและสามารถทำการจองได้

Use Case Name	View Route Station
Actor	USER
Descriptions	ค้นหาและแสดงเส้นทางพร้อมตารางเวลาเดินรถ จากสถานีต้นทางไปยังสถานีปลายทางที่ผู้ใช้ระบุ

Use Case Name	View Route Trip
Actor	USER
Descriptions	ดูรายละเอียดของรอบการเดินทาง (Trip) แบบเจาะจง ระหว่างสองสถานีในวันและเวลาที่กำหนด โดยจะแสดงสถานีปลายทางที่วิ่งผ่าน ข้อมูลขบวนรถไฟ และสถานะความว่างของที่นั่งทั้งหมด

Customer

Use Case Name	Register
Actor	Customer
Descriptions	ลงทะเบียนลูกค้าใหม่เข้าสู่ระบบ โดยใช้ชื่อ อีเมล และเบอร์โทรศัพท์ และจะส่งคืนรหัสลูกค้า (Customer ID) พร้อมรายละเอียดข้อมูลที่ลงทะเบียน

Use Case Name	View Profile
Actor	Customer
Descriptions	ดูข้อมูลโปรไฟล์พื้นฐานของลูกค้า เช่น รหัสลูกค้า, ชื่อ, อีเมล, เบอร์โทรศัพท์ และคะแนนสะสม (Reward Points)

Use Case Name	Update Profile
Actor	Customer
Descriptions	อัปเดตข้อมูลส่วนตัวของลูกค้า ได้แก่ ชื่อ, อีเมล และเบอร์โทรศัพท์

Use Case Name	View Booking
Actor	Customer
Descriptions	แสดงรายการ "การจอง" ทั้งหมดของลูกค้า พร้อมรายละเอียดอย่างครบถ้วน เช่น รหัสการจอง, วันเวลา, สถานีต้นทาง-ปลายทาง, ข้อมูลผู้โดยสารและที่นั่ง, ราคา และสถานะปัจจุบัน (เช่น PENDING, PAID, CANCELLED)

Use Case Name	View Ticket
Actor	Customer
Descriptions	แสดงรายการ "ตั๋วโดยสาร" ที่ยืนยันแล้วทั้งหมดของลูกค้า รวมถึงรหัสตัว, รายละเอียดเส้นทาง, รถไฟ, ที่นั่ง, ราคา และสถานะตั๋ว

Use Case Name	Add Booking
Actor	Customer
Descriptions	สร้างการจองตัวใหม่สำหรับลูกค้า ในรอบการเดินทาง ตั้โดยสาร และที่นั่งที่ระบุ (สถานะเริ่มต้นหลังทำรายการคือ PENDING)

Use Case Name	Pay Booking
Actor	Customer
Descriptions	ชำระเงินสำหรับการจองที่ค้างชำระ (PENDING) โดยใช้รหัสอ้างอิงการชำระเงิน เมื่อสำเร็จจะสร้างตั๋ว (Ticket) ที่ยืนยันแล้ว

Use Case Name	Pay Booking With Point
Actor	Customer
Descriptions	ชำระค่าตั๋วโดยสารโดยใช้คะแนนสะสม (Reward Points) ของลูกค้าแทนการจ่ายเงิน

Use Case Name	Change Booking
Actor	Customer
Descriptions	เปลี่ยนแปลงวันที่ของการจอง โดยรักษาสถานีต้นทาง-ปลายทาง และตำแหน่งที่นั่งเดิมไว้สำหรับรอบการเดินทางในวันใหม่

Use Case Name	Cancel Booking
Actor	Customer
Descriptions	ยกเลิกการจองตัวของลูกค้า และคืนที่นั่งที่ถูกจองไว้กลับเข้าสู่ระบบเพื่อให้คนอื่นจองได้

Use Case Name	Refund Booking
Actor	Customer
Description	ขอคืนเงินสำหรับการจองที่ชำระเงินไปแล้ว ระบบจะจัดการคืนเงินเข้าช่องทางเดิม ยกเลิกตั๋ว และคืนที่นั่งกลับสู่ระบบ

Use Case Name	Buy Food
Actor	Customer
Description	ขอคืนเงินสำหรับการจองที่ชำระเงินไปแล้ว ระบบจะจัดการคืนเงินเข้าช่องทางเดิม ยกเลิกตั๋ว และคืนที่นั่งกลับสู่ระบบ

StationOfficer

Use Case Name	Enter Trip
Actor	StationOfficer
Descriptions	บันทึกสถานะเมื่อรถไฟในรอบการเดินทาง (Trip) วิ่งเข้าเทียบชานชาลาที่สถานี

Use Case Name	Exit Trip
Actor	StationOfficer
Descriptions	บันทึกสถานะเมื่อรถไฟในรอบการเดินทาง (Trip) วิ่งออกจากสถานี

Use Case Name	Verify Ticket
Actor	StationOfficer
Descriptions	ตรวจสอบตั๋วของลูกค้าก่อนขึ้นรถไฟ เพื่อยืนยันว่าตั๋วถูกต้องตรงกับลูกค้า สถานี และเวลาเดินทาง หากตรวจสอบผ่านจะอัปเดตสถานะตั๋วเป็น 'USED' (ใช้งานแล้ว)

Use Case Name	View History
Actor	StationOfficer, SystemAdministrator
Descriptions	ตรวจสอบประวัติการปฏิบัติงาน (Usage history) ของเจ้าหน้าที่แต่ละราย

SystemAdministrator

Use Case Name	View All Route
Actor	SystemAdministrator
Descriptions	แสดงข้อมูลเส้นทางทั้งหมดในระบบ รวมถึงเส้นทางที่ถูกกระทำการใช้งานไปแล้ว เพื่อใช้ในการตรวจสอบ

Use Case Name	View All Train
Actor	SystemAdministrator
Descriptions	แสดงข้อมูลขบวนรถไฟทั้งหมดในระบบ

Use Case Name	View All Trip
Actor	SystemAdministrator
Descriptions	แสดงข้อมูลรอบการเดินทาง (Trip) ทั้งหมด รวมถึงจำนวนความจุที่นั่งและยอดการจอง

Use Case Name	Add Route
Actor	SystemAdministrator
Descriptions	สร้างเส้นทางเดินรถใหม่ โดยกำหนดสถานีต้นทาง สถานีปลายทาง และระยะทางระหว่างสถานี

Use Case Name	Remove Route
Actor	SystemAdministrator
Descriptions	ระงับการใช้งาน (Decommission) เส้นทาง ทำให้ไม่สามารถจองได้อีก พร้อมกับยกเลิกรอบการเดินทางที่เกี่ยวข้องกับเส้นทางนี้ทั้งหมดโดยอัตโนมัติ

Use Case Name	Add Schedule
Actor	SystemAdministrator
Descriptions	สร้างตารางเวลาเดินรถเปล่า สำหรับนำไปผูกกับเส้นทางที่ระบุ

Use Case Name	Add Schedule Time
Actor	SystemAdministrator
Descriptions	เพิ่มเวลาออกเดินทาง (ชั่วโมง:นาที) เข้าไปในตารางเวลาของเส้นทาง

Use Case Name	Add Train
Actor	SystemAdministrator
Descriptions	เพิ่มขบวนรถไฟใหม่เข้าสู่ระบบ โดยกำหนดคลาสของรถไฟ (Commuter, Speeder, Splinter) และจำนวนตู้โดยสารปกติ/ธุรกิจ

Use Case Name	Remove Train
Actor	SystemAdministrator
Descriptions	ระงับการใช้งานรถไฟขบวนที่ระบุ ซึ่งจะส่งผลให้รอบการเดินทาง (Trip) ที่ใช้รถไฟขบวนนี้ถูกยกเลิกไปด้วย

Use Case Name	Add Trip
Actor	SystemAdministrator
Descriptions	อนุมัติสร้างรอบการเดินทาง (Trip) ใหม่เข้าสู่ระบบเพื่อให้ลูกค้าจอง โดยอ้างอิงจากเส้นทางรถไฟวันที่เดินทาง และเวลาจากตาราง

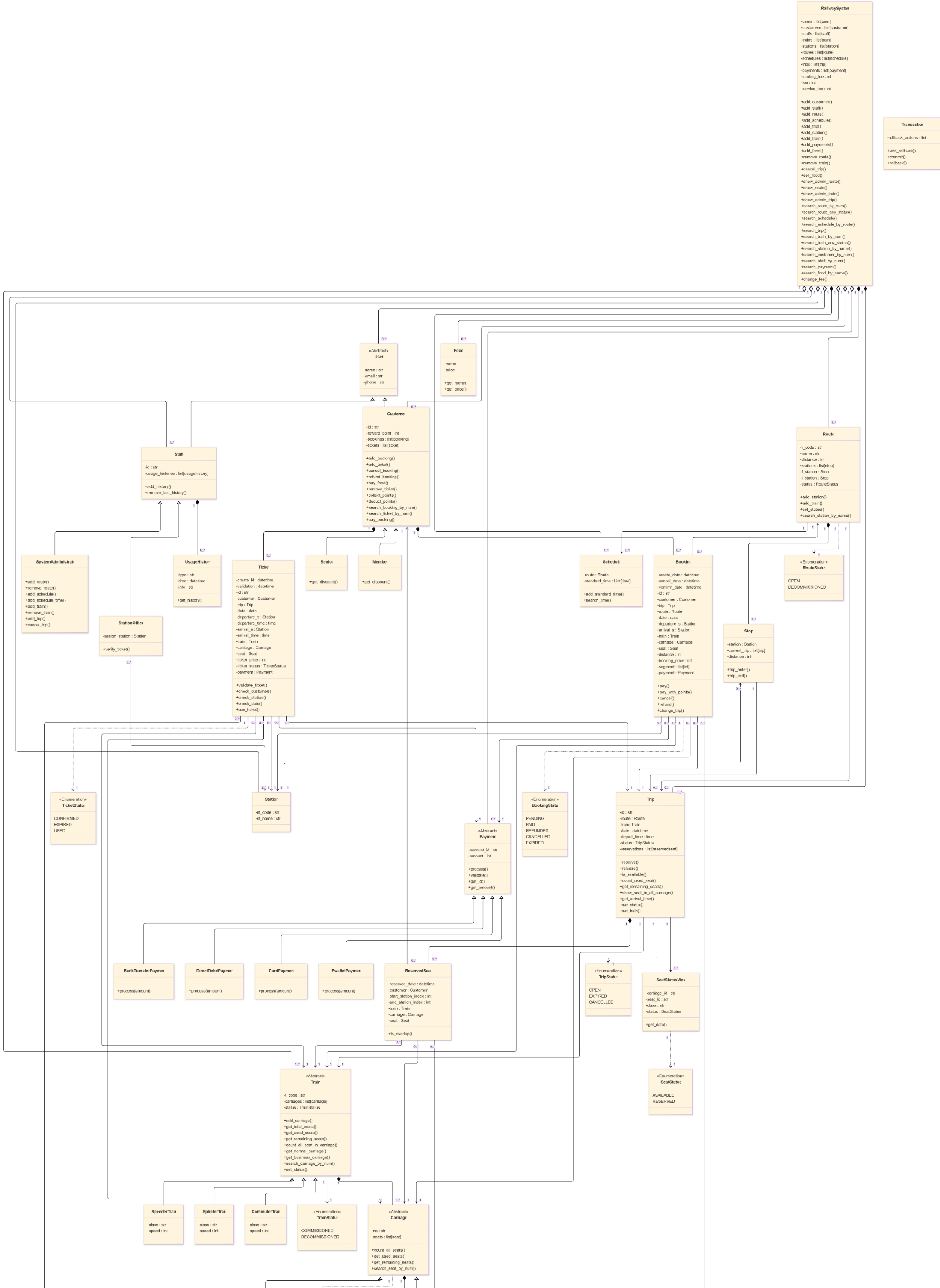
Use Case Name	Remove Trip
Actor	SystemAdministrator
Descriptions	ยกเลิกรอบการเดินทาง (Trip) ที่เปิดไปแล้ว หากมีการยกเลิก ระบบจะดำเนินการจัดการยกเลิกตัวที่เกี่ยวข้องด้วย

Use Case Name	Register Staff
Actor	SystemAdministrator
Descriptions	สร้างบัญชีสำหรับพนักงานใหม่ โดยสามารถกำหนดตำแหน่งเป็นเจ้าหน้าที่สถานี (ระบุสถานีที่ประจำการ) หรือสร้างผู้ดูแลระบบคนใหม่

Use Case Name	Change Fee
Actor	SystemAdministrator
Description	ปรับปรุงโครงสร้างราคาของระบบรถไฟ ได้แก่ ค่าธรรมเนียมเริ่มต้น (Starting fee), อัตราค่าโดยสารตามระยะทาง (Fee) และค่าบริการเพิ่มเติม (Service fee)

บทที่ 3 Class Diagram

Link Mermaid: <https://mermaid.ai/d/732bdeef-933c-4c64-a544-95a437a91d75>



บทที่ 4 รายละเอียดการทำงาน

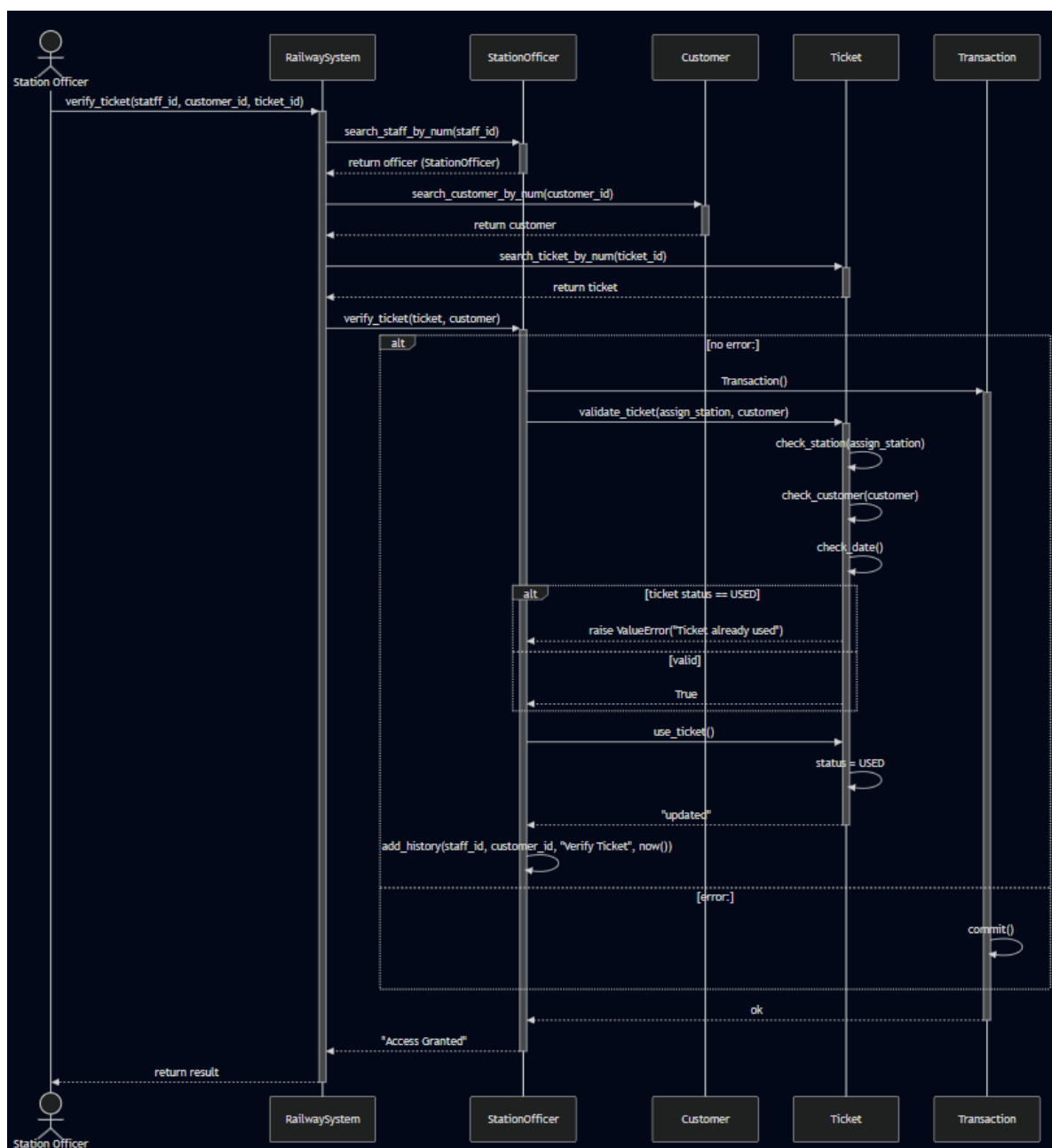
ชื่อ: Verify Ticket

หน้าที่: ใช้โดยเจ้าหน้าที่ประจำสถานีเพื่อตรวจตั๋วโดยสารของผู้โดยสาร

พารามิเตอร์: ส่ง staff_id, customer_id, ticket_id

เซอร์วิสที่เกิดขึ้น: เป็นการตรวจตั๋วโดยสารโดยจะแสดงผลเป็น ticket_id, status, result, route, date, seat, verified by staff_id

Sequence Diagram:



Actor: StationOfficer

Flow: 1. StationOfficer ส่ง staff_id, customer_id, ticket_id เข้ามาที่ระบบ

2. ระบบตรวจสอบสิทธิของ StationOfficer ถ้าไม่มีสิทธิจะปฏิเสธ

3. ระบบจะส่ง object StationOfficer ให้ตรวจสอบตั๋วผ่านเมธอด verify_ticket

4. ตรวจสอบสถานะปัจจุบันของตั๋วว่าเป็น CONFIRMED ตรวจสอบวันที่ ลูกค้าและสถานีว่าถูกต้องตามที่กำหนดหรือไม่

5. หากข้อมูลถูกต้อง สถานะของตั๋วจะกลายเป็น USED

6. ถ้ามีข้อผิดพลาดระบบจะย้อนกลับ

7. StationOfficer ลงบันทึกการทำงานเข้าระบบ UsageHistory

8. ระบบส่งข้อความตอบกลับไปยัง Actor ว่าตรวจสอบสำเร็จ

Code ใน Class:

```
def search_customer_by_num(self, num):
    for customer in self.__customers:
        if customer.get_user_id().lower() == num.lower():
            return customer
    raise KeyError("Customer not found")

def search_staff_by_num(self, num):
    for staff in self.__staffs:
        if staff.get_user_id().lower() == num.lower():
            return staff
    raise KeyError("Staff not found")
```

```
def search_ticket_by_num(self, num):
    for ticket in self.__tickets:
        if ticket.get_ticket_id() == num: return ticket
    raise KeyError("Ticket not found")
```

```
def verify_ticket(self, ticket, customer):
    trans = Transaction()
    try:
        old_status = ticket.get_ticket_status()

        trans.add_rollback(lambda: ticket.set_ticket_status(old_status) if ticket.get_ticket_status() != TicketStatus.EXPIRED else None)
        ticket.validate_ticket(self.__assign_station, customer)
        ticket.use_ticket()

        self.add_history(self.get_user_id(), customer.get_user_id(), 'Verify Ticket', datetime.now())
        trans.add_rollback(lambda: self.remove_last_history())
        trans.commit()
    except Exception as e:
        trans.rollback()
        raise e
    return "Access Granted"
```

Code ใน MCP:

```
@mcp.tool()
def verify_a_ticket(staff_id: str, customer_id: str, ticket_id: str):
    """verify Ticket"""
    try:
        staff = arl.search_staff_by_num(staff_id)
        if not isinstance(staff, StationOfficer): raise ValueError("Unauthorized")
        customer = arl.search_customer_by_num(customer_id)
        ticket_found = customer.search_ticket_by_num(ticket_id)
        result = staff.verify_ticket(ticket_found, customer)
        return { f"Message : {result}",
                f"Ticket Id : {ticket_found.get_ticket_id()}",
                f"Departure Date : {ticket_found.get_ticket_date().strftime('%d/%m/%Y %H:%M')}",
                f"Ticket Status: {ticket_found.get_ticket_status().value}"
              }
    except Exception as e:
        return f"เกิดข้อผิดพลาด: {str(e)}"
```

Prompt: ตอนนี้ผมเป็นเจ้าหน้าที่ S1 ทำการตรวจตั๋วของ ลูกค้า C1

Orchestrated ticket verification workflow for customer and staff >

ต้องดูตั๋วของลูกค้า C1 ก่อนครับ

Used train-project integration >

ตรวจตั๋วเรียบร้อยแล้วครับ! ☒

รายละเอียด	ข้อมูล
Ticket ID	T2
สถานะ	☒ USED
ผล	Access Granted
เส้นทาง	พระจอมเกล้า → หัวลำโพง
วันที่เดินทาง	09/03/2026
ที่นั่ง	Ca1 - 1A
ตรวจโดย	เจ้าหน้าที่ S1

ตัวผ่านการตรวจสอบแล้วครับ 

    < 2/2 >

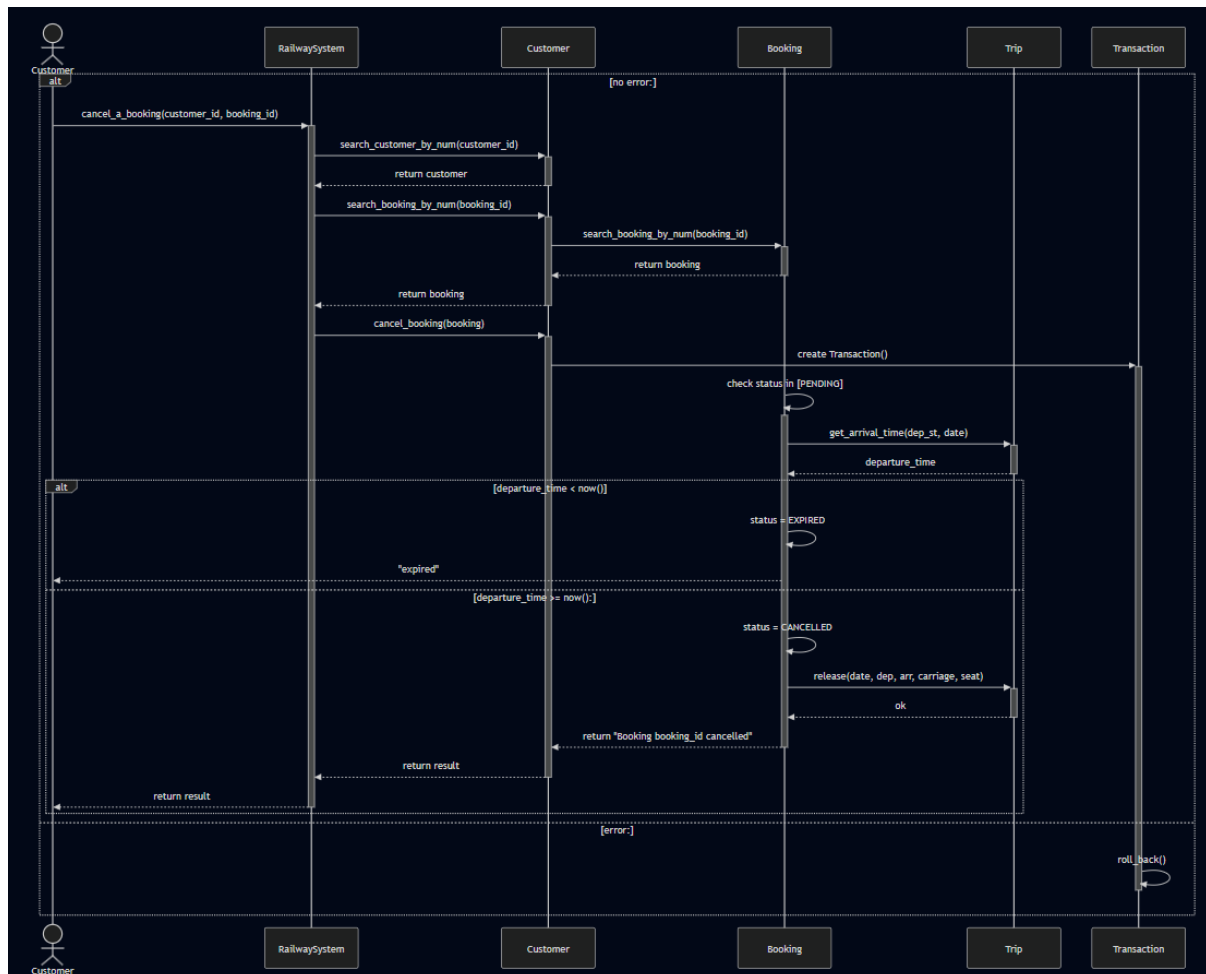
ชื่อ: Cancel Booking

หน้าที่: ใช้โดยลูกค้า เพื่อยกเลิกการจองที่จองไปก่อนหน้านี้

พารามิเตอร์: ส่ง customer_id, booking_id

เซอร์วิสที่เกิดขึ้น: เป็นการยกเลิกตั๋วที่ create มาก่อนหน้านี้ โดยคืนมาเป็น booking_id และบอกว่ายกเลิกแล้ว

Sequence Diagram:



Actor: Customer

- Flow: 1. Customer ส่งยกเลิกการจองที่ระบบโดยระบุ customer_id, booking_id
2. ระบบค้นหา object ลูกค้า และหา object การจองที่เก็บในลูกค้า
3. ระบบส่ง object Customer ให้ยกเลิกการจองโดย object การจอง
4. ตรวจสอบสถานะปัจจุบันของการจองว่าเป็น PENDING ตรวจสอบวันที่ ว่าเกินกำหนดหรือยัง
5. หากข้อมูลถูกต้อง สถานะของการจองจะกลายเป็น CANCELLED
6. รบรยกเลิกการจองเก้าอี้ โดย date, dep_station, arr_station, carriage, seat
7. ถ้ามีข้อผิดพลาดระบบจะย้อนกลับ

8. ระบบส่งข้อความตอบกลับไปยัง Actor ว่าตรวจสอบสำเร็จ

Code ใน Class:

```
def search_customer_by_num(self, num):
    for customer in self.__customers:
        if customer.get_user_id().lower() == num.lower():
            return customer
    raise KeyError("Customer not found")

def search_staff_by_num(self, num):
    for staff in self.__staffs:
        if staff.get_user_id().lower() == num.lower():
            return staff
    raise KeyError("Staff not found")
```

```
def search_booking_by_num(self, num):
    for booking in self.__bookings:
        if booking.get_booking_id() == num: return booking
    raise KeyError("Booking not found")
```

```
def cancel_booking(self, booking):
    trans = Transaction()
    try:
        result = booking.cancel()
        trans.commit()
    except Exception as e:
        trans.rollback()
        raise e
    return result
```

```
def cancel(self):
    dep, arr = self.__segment
    if self.__booking_status not in [BookingStatus.PENDING, BookingStatus.PAID]:
        raise ValueError("Cannot cancel booking")
    departure_time = self.__trip.get_arrival_time(self.__departure_s, self.__date.date())
    if departure_time < datetime.now():
        self.__booking_status = BookingStatus.EXPIRED
        result = "expired"
    else:
        self.__booking_status = BookingStatus.CANCELLED
        result = f"Booking Id: {self.__id} cancelled"
    self.__cancel_date = datetime.now()
    self.__trip.release(self.__date.date(), dep, arr, self.__carriage, self.__seat)
    return result
```

Code ใน MCP:

```
@mcp.tool()
def cancel_a_booking(customer_id: str, booking_id:str):
    """Cancel Booking"""
    try:
        customer = arl.search_customer_by_num(customer_id)
        booking = customer.search_booking_by_num(booking_id)
        result = customer.cancel_booking(booking)
        return{ f"Message : {result}, Booking Id : {booking.get_booking_id()}" }
    except Exception as e:
        return f"เกิดข้อผิดพลาด: {str(e)}"
```

Prompt: ตอนนี้ผมเป็นลูกค้า C2 ต้องการยกเลิกการจองที่มี

กำลังพิจารณาการยกเลิกการจองของลูกค้า C2 >

ยกเลิกการจองเรียบร้อยแล้วครับ! 

รายละเอียด	ข้อมูล
Booking ID	B2
สถานะ	 CANCELLED
เส้นทาง	พระจอมเกล้า → หัวลำโพง
วันที่	09/03/2026
ที่นั่ง	Ca1 - 1B
ราคา	167.50 บาท

การจอง B2 ถูกยกเลิกแล้วครับ

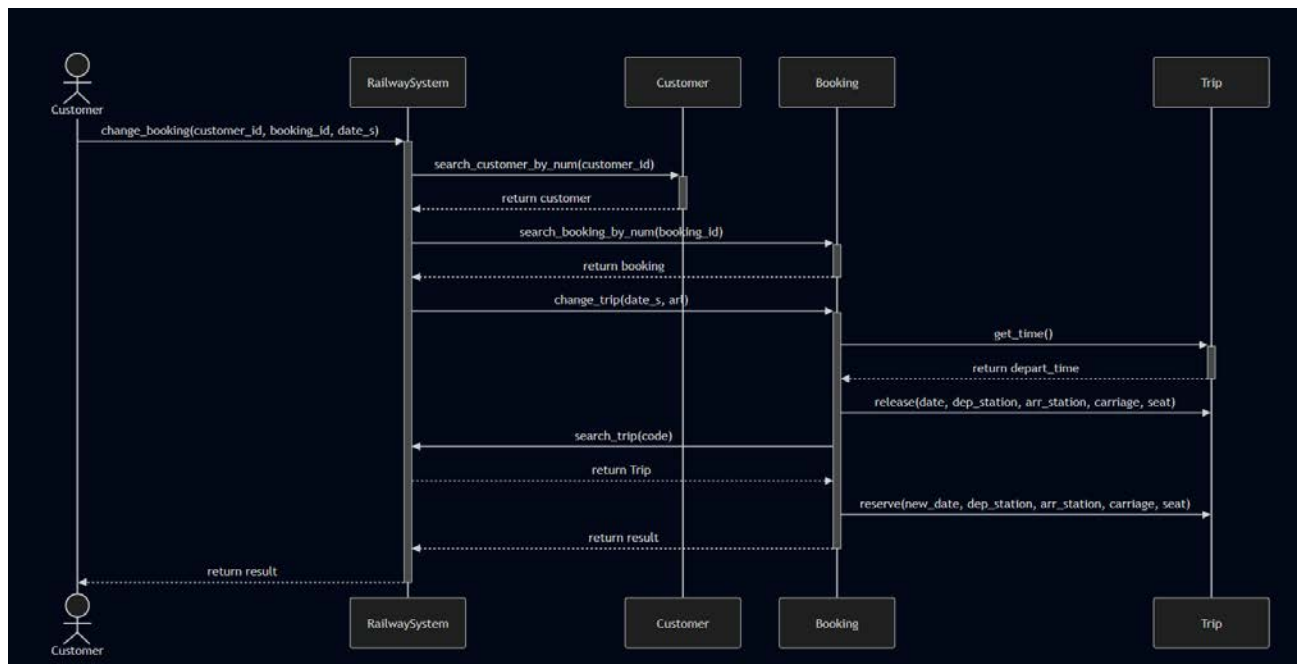
ชื่อ: Change Booking

หน้าที่: ใช้โดยลูกค้า เพื่อเปลี่ยนวันที่การจองโดยเปลี่ยนแค่รอบรถ

พารามิเตอร์: ส่ง customer_id, booking_id, date

เซอร์วิสที่เกิดขึ้น: เป็นการเปลี่ยนวันที่ของการจองโดยคืน booking_id วันที่ก่อนเปลี่ยน และวันที่หลังเปลี่ยน

Sequence Diagram:



Actor: Customer

- Flow: 1. Customer ส่งเปลี่ยนวันที่การจองที่ระบบโดยระบุ customer_id, booking_id, วันที่ใหม่
2. ระบบค้นหา object ลูกค้า และหา object การจองที่เก็บในลูกค้า
3. ระบบสั่ง object Booking ให้เปลี่ยนการจองโดย object การจอง
4. Object Booking เรียกดูเวลาก่อนเปลี่ยนจาก object Trip
5. Object Booking สั่ง Trip ให้ปล่อยการจองของเวลาก่อนหน้าออก
6. Object Booking ค้นหา Trip ด้วยรหัสรอบรถของวันที่ใหม่
7. Object Booking จองเก้าอี้ใหม่โดย reserve(new_date, dep_station, arr_station, carriage, seat)
8. ระบบคืนค่ากลับไปให้ Actor โดยแจ้ง Booking_id วันที่เก่า และวันที่ใหม่

Code ใน Class:

```
def search_customer_by_num(self, num):
    for customer in self.__customers:
        if customer.get_user_id().lower() == num.lower():
            return customer
    raise KeyError("Customer not found")

def search_staff_by_num(self, num):
    for staff in self.__staffs:
        if staff.get_user_id().lower() == num.lower():
            return staff
    raise KeyError("Staff not found")
```

```
def search_booking_by_num(self, num):
    for booking in self.__bookings:
        if booking.get_booking_id() == num: return booking
    raise KeyError("Booking not found")
```

```
def change_trip(self, date_s, system):
    dep, arr = self.__segment
    new_date = datetime.strptime(date_s, "%Y-%m-%d")
    depart_time = self.__trip.get_time()
    before_trip = self.__trip
    self.__trip.release(self.__date.date(), dep, arr, self.__carriage, self.__seat)
    code = f"{self.__route.get_route_id()}-{new_date.strftime('%Y%m%d')}-{depart_time.strftime('%H%M')}"
    self.__date = new_date
    trip = system.search_trip(code)
    self.__trip = trip
    self.__trip.reserve(self.__date.date(), self.__customer, dep, arr, self.__carriage, self.__seat)

    result = f"Booking Id: {self.__id} | before : {before_trip.get_trip_id()} | after : {self.__trip.get_trip_id()}"
    return result
```

Code ใน MCP:

```
@mcp.tool()
def change_booking(customer_id: str, booking_id: str, date_s: str):
    """Change date in booking"""
    try:
        customer = arl.search_customer_by_num(customer_id)
        booking = customer.search_booking_by_num(booking_id)

        result = booking.change_trip(date_s, arl)

        return {"Message": result}
    except Exception as e:
        return f"เกิดข้อผิดพลาด: {str(e)}"
```

Prompt: ในฐานะ c2 ขอเปลี่ยนวันการจองเป็นวันมะรืน

ดูการจองของ C2 ก่อนเลยครับ!

[View customer bookings >](#)

พบ Booking B2 วันที่ 10/03/2026 — วันมะรืนคือ 11/03/2026 ครับ ดำเนินการเปลี่ยนวันเลย!

[Used demo integration >](#)

เปลี่ยนวันเรียบร้อยแล้วครับ! ✅

สรุปการเปลี่ยนแปลง:

รายการ	รายละเอียด
📅 Booking	B2
📅 วันเดิม	10/03/2026
📅 วันใหม่	11/03/2026 (วันมะรืน)
📍 เส้นทาง	Phrajomklao → Hualamphong เวลา 08:30 น.

📄 👍 💬 ↺

Actor: Customer

Flow:

1. Customer ส่ง customer_id, date, depart_time, carriage_id, seat_id, departure_station_name, arrival_station_name เข้ามาที่ระบบ
2. ระบบค้นหาข้อมูล Customer, สถานีต้นทาง และสถานีปลายทางจาก RailwaySystem
3. ระบบค้นหา Schedule ที่ตรงกับเส้นทางและเวลา แล้วดึง Route และค้นหา Trip ที่ตรงกัน
4. ระบบค้นหาขบวนรถจาก Trip ถ้าไม่พบขบวนรถจะปฏิเสธการจอง
5. ระบบค้นหาตู้โดยสารจาก Train และค้นหาที่นั่งจาก Carriage
6. ระบบตรวจสอบว่าเวลาจองยังไม่ผ่านมา ถ้าเลยเวลาแล้วจะปฏิเสธการจอง
7. ระบบสร้าง Booking object โดยคำนวณราคาจากระยะทาง ประเภทตู้โดยสาร และส่วนลด พร้อมจองที่นั่งใน Trip
8. ระบบบันทึก Booking เข้าสู่ Customer ด้วย add_booking()
9. ระบบส่งข้อมูลตอบกลับไปยัง Customer ว่าจองสำเร็จ พร้อม booking_id, เส้นทาง, เวลา, ราคา และ status = PENDING

Code ที่เกี่ยวข้อง

```
class Booking:
    __booking_id_counter = 1
    def __init__(self, customer, trip, date, departure_s, arrival_s, train, carriage, seat, system):
        self.__create_date = datetime.now()
        self.__cancel_date = None
        self.__confirm_date = None
        self.__id = "B" + str(Booking.__booking_id_counter)
        Booking.__booking_id_counter += 1
        self.__customer = customer
        self.__trip = trip
        self.__route = trip.get_route()
        self.__date = date
        self.__departure_s = departure_s
        self.__arrival_s = arrival_s
        self.__train = train
        self.__carriage = carriage
        self.__seat = seat
        self.__distance = self.__route.get_stop_distance(departure_s, arrival_s)
        self.__booking_price = (((self.__distance * system.get_fee() + system.get_starting_fee()) * carriage.get_carriage_fee()) * customer.get_discount()) + system.get_s
        self.__booking_status = BookingStatus.PENDING
        dep = self.__route.get_stop_index(departure_s)
        arr = self.__route.get_stop_index(arrival_s)
        if dep >= arr: raise ValueError("Invalid direction")
        self.__segment = (dep, arr)
        self.__trip.reserve(self.__date.date(), self.__customer, dep, arr, self.__carriage, self.__seat)
        self.__payment = None
```

```

class Customer(User):
    __customer_id_counter = 1
    def __init__(self, name, email, phone):
        super().__init__(name, email, phone)
        self.__id = "C" + str(Customer.__customer_id_counter)
        Customer.__customer_id_counter += 1
        self.__reward_point = 0
        self.__bookings = []
        self.__tickets = []

    def add_booking(self, booking):
        self.__bookings.append(booking)

```

```

class Trip:
    def __init__(self, route, train, date, depart_time):
        self.__id = f"{route.get_route_id()}-{date.strftime('%Y%m%d')}-{depart_time.strftime('%H%M')}"
        self.__route = route
        self.__train = train
        self.__date = date
        self.__depart_time = depart_time
        self.__status = TripStatus.OPEN
        self.__reservations = []

    def reserve(self, date, customer, start, end, carriage, seat):
        if start >= end: raise ValueError("Invalid segment")
        if not self.is_available(date, start, end, carriage, seat):
            raise ValueError("Already reserved")
        reservation = ReservedSeat(date, customer, start, end, self.__train, carriage, seat)
        self.__reservations.append(reservation)

```

```

@sup.tool()
def add_a_booking(customer_id: str, date_s: str, depart_time: str, carriage_id: str, seat_id: str, departure_station_name: str, arrival_station_name: str):
    """Booking for customer"""
    try:
        customer = ari.search_customer_by_num(customer_id)
        date = datetime.strptime(date_s, "%Y-%m-%d")
        time = datetime.strptime(depart_time, "%H:%M")
        dep_st = ari.search_station_by_name(departure_station_name)
        arr_st = ari.search_station_by_name(arrival_station_name)
        schedule = ari.search_schedule(dep_st, arr_st, time.time())
        route = schedule.get_route()
        search_t = f"{route.get_route_id()}-{datetime.strftime(date, '%Y%m%d')}-{datetime.strftime(time, '%H%M')}"
        trip = ari.search_trip(search_t)
        train = trip.get_train()
        if train == None: raise Exception("Invalid no train available")
        carriage = train.search_carriage_by_num(carriage_id)
        seat = carriage.search_seat_by_num(seat_id)

        booking_time = datetime.combine(date.date(), trip.get_time())
        if booking_time < datetime.now(): raise ValueError("Cannot book past trip")

        booking = Booking(customer, trip, booking_time, dep_st, arr_st, train, carriage, seat, ari)
        customer.add_booking(booking)
        return {
            "Customer id": (customer.get_user_id()),
            "Booking id": (booking.get_booking_id()),
            "Route": (booking.get_booking_route().get_route_name()),
            "Departure Station": (booking.get_booking_departure().get_station_name()),
            "Departure Time": (booking.get_booking_trip().get_arrival_time(dep_st, booking_time.date()).strftime('%d/%m/%Y %H:%M')),
            "Arrival Station": (booking.get_booking_trip().get_station_name()),
            "Arrival time": (booking.get_booking_trip().get_arrival_time(arr_st, booking_time.date()).strftime('%d/%m/%Y %H:%M')),
            "Price": (booking.get_booking_price())
        }
    except Exception as e:
        return f"เกิดข้อผิดพลาด: {str(e)}"

```

Prompt: ผมจะจองตั๋วจากลาดกระบังไปอโศก รอบ 8:31 ของวันพรุ่งนี้ ขอเป็นที่นั่งธรรมดา ถ้ามีจองให้ได้เลย

กำลังพิจารณาการจองตั๋วรถไฟจากลาดกระบังไปอโศก >

จองตั๋วเรียบร้อยแล้วครับ! 🎉

รายละเอียด	ข้อมูล
Booking ID	B4
ผู้โดยสาร	james (C4)
เส้นทาง	พระจอมเกล้า → หัวลำโพง (R1)
ต้นทาง	ลาดกระบัง — 08:31
ปลายทาง	อโศก — 08:43
วันที่	09/03/2026
ที่นั่ง	Ca1 - 1D (Normal)
ราคา	135 บาท

กรุณาชำระเงินเพื่อยืนยันการจองด้วยครับ! ➡

📄 🗂️ 🔄 🔄

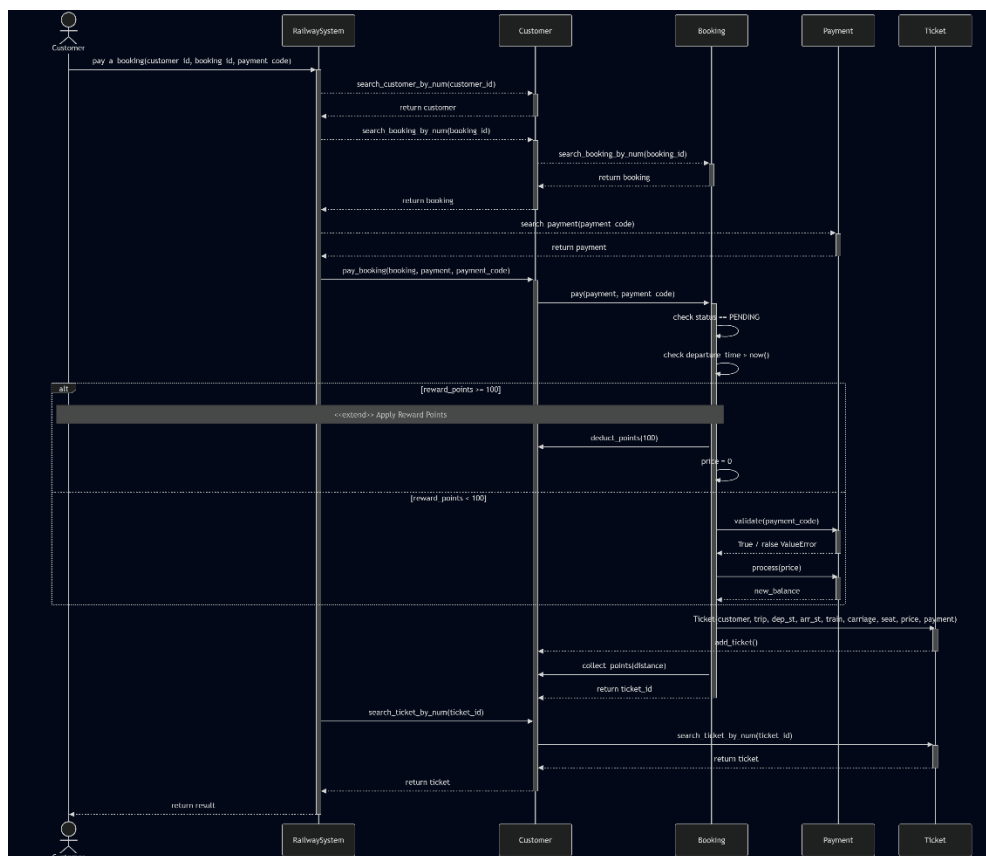
ชื่อ: Pay a Booking with Payment

หน้าที่: ใช้โดย Customer เพื่อชำระเงินค่าจองตั๋วรถไฟด้วยวิธีการชำระเงิน เช่น Card หรือ E-Wallet

พารามิเตอร์: ส่ง customer_id, booking_id, payment_code

เชอร์วิสที่เกิดขึ้น: เป็นการชำระเงินและออกตั๋วโดยระบบจะแสดงผลเป็น ticket_id, route, departure, arrival, seat, price, issue_date, status = CONFIRMED

Sequence diagram:



Actor: Customer

Flow:

1. Customer ส่ง customer_id, booking_id, payment_code เข้ามาที่ระบบผ่าน pay_a_booking()
2. ระบบค้นหาข้อมูล Customer ด้วย search_customer_by_num(customer_id) แล้วค้นหา Booking ด้วย search_booking_by_num(booking_id) จาก Customer และค้นหา Payment ด้วย search_payment(payment_code) จาก RailwaySystem
3. ระบบเรียก pay_booking(booking, payment, payment_code) บน Customer ซึ่งส่งต่อไปยัง pay(payment, payment_code) บน Booking
4. Booking ตรวจสอบว่า status เป็น PENDING และ departure_time ยังไม่ผ่านเวลาปัจจุบัน

5. `alt [reward_points >= 100]`: หาก Customer มี Reward Points เพียงพอ ระบบจะ extend ไปยัง Apply Reward Points โดยเรียก `deduct_points(100)` บน Customer และตั้งค่า `price = 0`
6. `alt [reward_points < 100]`: หาก Customer มี Reward Points ไม่เพียงพอ ระบบจะเรียก `validate(payment_code)` บน Payment เพื่อตรวจสอบความถูกต้อง ถ้าไม่ผ่านจะ raise `ValueError` จากนั้นเรียก `process(price)` บน Payment เพื่อตัดยอดเงิน แล้วได้รับ `new_balance` กลับมา
7. Booking สร้าง Ticket object ด้วย `Ticket(customer, trip, dep_st, arr_st, train, carriage, seat, price, payment)` แล้วเรียก `add_ticket()` บน Customer เพื่อบันทึก Ticket
8. Booking เรียก `collect_points(distance)` บน Customer เพื่อสะสม Reward Points ตามระยะทาง แล้ว return `ticket_id` กลับมา
9. ระบบค้นหา Ticket ด้วย `search_ticket_by_num(ticket_id)` จาก Customer แล้วได้รับ ticket กลับมา
10. ระบบส่งข้อมูลตอบกลับไปยัง Customer ว่าชำระเงินสำเร็จ พร้อม `ticket_id`, เส้นทาง, เวลา, ราคา และยอดคงเหลือ

Code ที่เกี่ยวข้อง:

```
class Customer(User):

    def pay_booking(self, booking, payment, payment_code): return booking.pay(payment, payment_code)
```

```
class Booking:
    _booking_id_counter = 1
    def pay(self, payment: Payment, code):
        if self._booking_status != BookingStatus.PENDING:
            raise ValueError("Cannot pay")
        departure_time = self._trip.get_arrival_time(self._departure_s, self._date.date())
        if departure_time < datetime.now():
            self._booking_status = BookingStatus.EXPIRED
            raise ValueError("Booking expired")
        self._booking_status = BookingStatus.PAID
        price = self._booking_price
        if not payment.validate(code):
            raise ValueError(f"Invalid payment validation code")
        if payment.get_amount() < price:
            raise ValueError(f"Insufficient payment amount. Required: {price}")
        payment.process(price)
        self._payment = payment
        ticket = Ticket(self._customer, self._trip, self._departure_s, self._arrival_s, self._train, self._carriage, self._seat, price, payment)
        self._customer.add_ticket(ticket)
        self._customer.collect_points(self._distance)
        self._confirm_date = datetime.now()
        return ticket.get_ticket_id()
```

```

class Payment(ABC):
    def __init__(self, amount, id):
        self.__account_id = id
        self.__amount = amount
    @abstractmethod
    def process(self, amount):
        pass
    def validate(self, receive_id):
        if self.__account_id == receive_id:
            return True
        return False
    def get_id(self):
        return self.__account_id
    def get_amount(self):
        return self.__amount

```

```

class EWalletPayment(Payment):
    def process(self, amount):
        new_amount = (self.get_amount() - amount) * 0.9
        self.__Payment__amount = new_amount
        return f"EWallet:{self.get_id()}:{new_amount}"

class CardPayment(Payment):
    def process(self, amount):
        new_amount = self.get_amount() - amount
        self.__Payment__amount = new_amount
        return f"CARD:{self.get_id()}:{new_amount}"

class BankTransferPayment(Payment):
    def process(self, amount):
        new_amount = (self.get_amount() - amount) * 0.95
        self.__Payment__amount = new_amount
        return f"Bank:{self.get_id()}:{new_amount}"

class DirectDebitPayment(Payment):
    def process(self, amount):
        new_amount = (self.get_amount() - amount) * 0.85
        self.__Payment__amount = new_amount
        return f"Debit:{self.get_id()}:{new_amount}"

```

```

@mcp.tool()
def pay_a_booking(customer_id:str, booking_id:str, payment_code:str):
    """Pay booking with payment"""
    try:
        customer = arl.search_customer_by_num(customer_id)
        booking = customer.search_booking_by_num(booking_id)
        payment = arl.search_payment(payment_code)
        result = customer.pay_booking(booking, payment, payment_code)
        ticket = customer.search_ticket_by_num(result)
        return {
            f"Message : Booking has been paid",
            f"Created Date : {ticket.get_ticket_create_date().strftime('%d/%m/%Y %H:%M')}",
            f"Ticket Id : {ticket.get_ticket_id()}",
            f"Route : {ticket.get_ticket_route().get_route_name()}",
            f"Departure Station : {ticket.get_ticket_departure().get_station_name()}",
            f"Departure Time : {ticket.get_ticket_trip().get_arrival_time(ticket.get_ticket_departure(), ticket.get_ticket_date().date()).strftime('%d/%m/%Y %H:%M')}",
            f"Arrival Station : {ticket.get_ticket_arrival().get_station_name()}",
            f"Arrival Time : {ticket.get_ticket_trip().get_arrival_time(ticket.get_ticket_arrival(), ticket.get_ticket_date().date()).strftime('%d/%m/%Y %H:%M')}",
            f"Price : {ticket.get_ticket_price()}",
            f"Amount left : {payment.get_amount()}"
        )
    except Exception as e:
        return f"เกิดข้อผิดพลาด: {str(e)}"

```

Prompt :ชำระเงินเลยด้วยบัตร 123456

Thinking about concerns with this request >

ชำระเงินสำเร็จแล้วครับ! 

รายละเอียด	ข้อมูล
Ticket ID	T1
ผู้โดยสาร	James (C4)
เส้นทาง	พระจอมเกล้า → หัวลำโพง (R1)
ต้นทาง	ลาดกระบัง — 08:31
ปลายทาง	อโศก — 08:43
วันที่เดินทาง	09/03/2026
ที่นั่ง	Ca1 - 1D (Normal)
ราคา	135 บาท
วันที่ออกตั๋ว	08/03/2026

ขอใจเดินทางปลอดภัยครับ! 

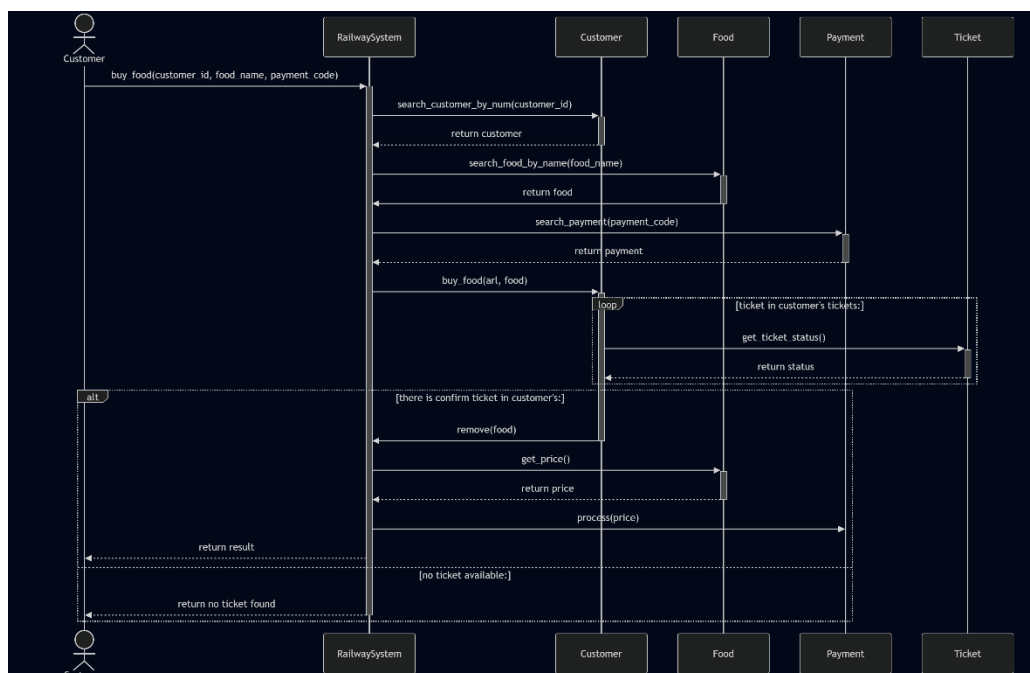
ชื่อ: Buy Food

หน้าที่: ใช้โดย Customer เพื่อสั่งซื้ออาหารบนขบวนรถไฟ โดย Customer จะต้องมียี่ห้อที่มีสถานะ CONFIRMED ก่อนจึงจะสามารถซื้ออาหารได้

พารามิเตอร์: ส่ง customer_id, food_name, payment_code

เซอวิซที่เกิดขึ้น: เป็นการซื้ออาหารและตัดยอดเงินจาก Payment โดยระบบจะแสดงผลเป็น food_name, food_price, balance_remaining

Sequence diagram:



Actor: Customer

Flow:

1. Customer ส่ง customer_id, food_name, payment_code เข้ามาที่ระบบผ่าน buy_food()
2. ระบบค้นหาข้อมูล Customer ด้วย search_customer_by_num(customer_id) แล้วค้นหาอาหาร ด้วย search_food_by_name(food_name) และค้นหา Payment ด้วย search_payment(payment_code) จาก RailwaySystem
3. ระบบเรียก buy_food(arl, food) บน Customer เพื่อเริ่มกระบวนการซื้ออาหาร
4. loop [ticket in customer's tickets:] Customer วนตรวจสอบ Ticket ทุกใบที่มีอยู่ โดยเรียก get_ticket_status() บน Ticket แต่ละใบ แล้วได้รับ status กลับมา เพื่อนับจำนวน Ticket ที่มีสถานะ CONFIRMED

5. alt [there is confirm ticket in customer's:] หาก Customer มี Ticket ที่สถานะ CONFIRMED อยู่ ระบบจะเรียก remove(food) บน RailwaySystem เพื่อนำอาหารออกจากระบบ จากนั้นเรียก get_price() บน Food เพื่อดึงราคาอาหาร แล้วเรียก process(price) บน Payment เพื่อตัดยอดเงินตามราคาอาหาร และ return result กลับไปยัง Customer
6. alt [no ticket available:] หาก Customer ไม่มี Ticket ที่สถานะ CONFIRMED ระบบจะ return "no ticket found" กลับไปยัง Customer

code ที่เกี่ยวข้อง:

```
class Food:
    def __init__(self, name, price):
        self.__name = name
        self.__price = price

    def get_name(self):
        return self.__name
    def get_price(self):
        return self.__price
```

```
class RailwaySystem:

    def add_food(self, food):
        self.__foods.append(food)
    def remove_food(self, food):
        for n_food in self.__foods:
            if n_food.get_food_name() == food.get_food_name():
                self.__foods.remove(food)
        raise KeyError("Food not found")

    def search_food_by_name(self, num):
        for food in self.__foods:
            if food.get_name() == num:
                return food
        raise KeyError("Food not found")
```

```
class Payment(ABC):

    def get_amount(self):
        return self.__amount
```

```
class Ticket:
    __ticket_id_counter = 1
    def get_ticket_status(self): return self.__ticket_status
```

Prompt: จ่ายเงินตัวที่ C1 มีอยู่ตอนนี้ด้วยบัตร 123456 และซื้อ Burger ด้วยเลยถ้าทำได้

ได้เลยครับ! ขอข้อมูล Booking ของ C1 ก่อนนะครับ

[View customer bookings >](#)

พบ Booking B1 ที่ยังค้างชำระอยู่ ดำเนินการจ่ายเงินและซื้อ Burger พร้อมกันเลยครับ!

[Used demo integration >](#)

ดำเนินการเสร็จเรียบร้อยแล้วครับ! 🎉

สรุปรายการ:

รายการ	รายละเอียด
💳 จ่ายส่ว Booking B1	Phrajomklao → Hualamphong, 185 บาท
🍔 ซื้อ Burger	50 บาท
ยอดรวมที่ไปจ่าย	235 บาท
คงเหลือในบัตร	1,765 บาท

ตัวได้รับ Ticket ID: T1 สำหรับเดินทางวันที่ 10/03/2026 เวลา 08:30 น. ครับ 🎫

📄 🗑️ 🗑️ 🗑️ < 5/5 >

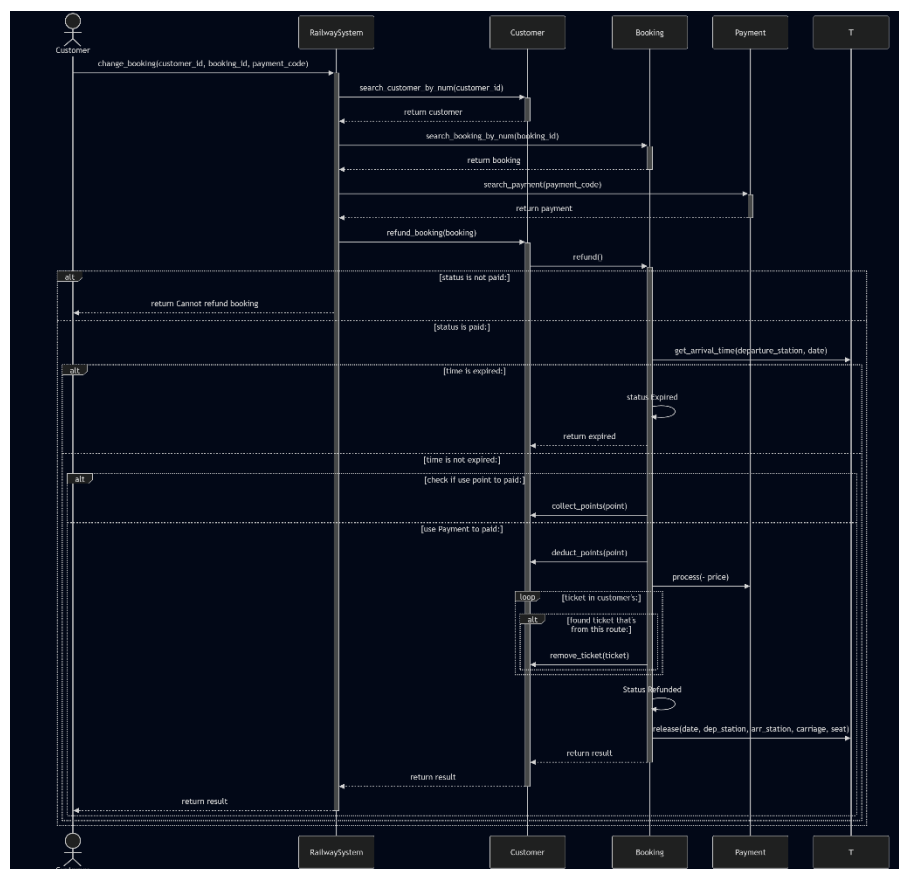
ชื่อ: Refund a Booking

หน้าที่: ใช้โดย Customer เพื่อขอคืนเงินสำหรับ Booking ที่ชำระเงินแล้ว โดยระบบจะคืนเงินกลับเข้าช่องทางการชำระเงินเดิม

พารามิเตอร์: ส่ง customer_id, booking_id, payment_code

เซอร์วิสที่เกิดขึ้น: เป็นการคืนเงินและยกเลิก Ticket โดยระบบจะแสดงผลเป็น booking_id, route, refund_amount, balance_remaining, status = REFUNDED

Sequence diagram:



Actor: Customer

Flow:

1. Customer ส่ง customer_id, booking_id, payment_code เข้ามาที่ระบบผ่าน change_booking()
2. ระบบค้นหาข้อมูล Customer ด้วย search_customer_by_num(customer_id) แล้วค้นหา Booking ด้วย search_booking_by_num(booking_id) และค้นหา Payment ด้วย search_payment(payment_code) จาก RailwaySystem
3. ระบบเรียก refund_booking(booking) บน Customer ซึ่งส่งต่อไปยัง refund() บน Booking
4. alt [status is not paid:] หาก Booking มีสถานะไม่ใช่ PAID ระบบจะ return "Cannot refund booking" กลับไปยัง Customer
5. alt [status is paid:] หาก Booking มีสถานะเป็น PAID ระบบจะเรียก get_arrival_time(departure_station, date) บน Trip เพื่อตรวจสอบเวลา
6. alt [time is expired:] หากเวลาเดินทางผ่านมาแล้ว Booking จะเปลี่ยนสถานะเป็น Expired และ return "expired" กลับมา
7. alt [time is not expired:] หากเวลาเดินทางยังไม่ผ่าน ระบบตรวจสอบวิธีการชำระเงินเดิมดังนี้
 - o alt [check if use point to paid:] หากชำระด้วย Point เดิม ระบบจะเรียก collect_points(point) บน Customer เพื่อคืน Point กลับ
 - o alt [use Payment to paid:] หากชำระด้วย Payment เดิม ระบบจะเรียก deduct_points(point) บน Customer เพื่อหัก Point ที่เคยสะสมคืน จากนั้นเรียก process(-price) บน Payment เพื่อคืนเงินกลับเข้าช่องทางชำระเงินเดิม
8. loop [ticket in customer's:] ระบบวนค้นหา Ticket ของ Customer โดย alt [found ticket that's from this route:] หากพบ Ticket ที่ตรงกับเส้นทางนี้ ระบบจะเรียก remove_ticket(ticket) บน Customer เพื่อลบ Ticket ออก
9. Booking เปลี่ยนสถานะเป็น REFUNDED และเรียก release(date, dep_station, arr_station, carriage, seat) บน Trip เพื่อคืนที่นั่งกลับสู่ระบบ
10. ระบบส่งข้อมูลตอบกลับไปยัง Customer ว่าคืนเงินสำเร็จ พร้อม booking_id, ยอดคืนเงิน และ ยอดคงเหลือ

Code ที่เกี่ยวข้อง:

```
class Booking:
    __booking_id_counter = 1
    def refund(self):
        dep, arr = self.__segment
        if self.__booking_status not in [BookingStatus.PAID]:
            raise ValueError("Cannot refund booking")
        departure_time = self.__trip.get_arrival_time(self.__departure_s, self.__date.date())
        if departure_time < datetime.now():
            self.__booking_status = BookingStatus.EXPIRED
            result = "expired"
        else:
            if self.__booking_price == 0:
                self.__customer.collect_points(100)
            else:
                self.__customer.deduct_points(self.__distance)
                self.__payment.process(-self.__booking_price)
                for ticket in list(self.__customer.get_user_tickets()):
                    if ticket.get_ticket_customer().get_user_id() == self.__customer.get_user_id():
                        if ticket.get_ticket_departure().get_station_name() == self.__departure_s.get_station_name():
                            if ticket.get_ticket_arrival().get_station_name() == self.__arrival_s.get_station_name():
                                self.__customer.remove_ticket(ticket)
                self.__booking_status = BookingStatus.REFUNDED
                result = f"Booking Id: {self.__id} refunded"
            self.__cancel_date = datetime.now()
            self.__trip.release(self.__date.date(), dep, arr, self.__carriage, self.__seat)
        return result
```

```
class Customer(User):
    def refund_booking(self, booking):
        trans = Transaction()
        try:
            result = booking.refund()
            trans.commit()
        except Exception as e:
            trans.rollback()
            raise e
        return result
```

```
@mcp.tool()
def refund_a_booking(customer_id: str, booking_id: str, payment_code: str):
    """Refund already paid booking"""
    try:
        customer = arl.search_customer_by_num(customer_id)
        booking = customer.search_booking_by_num(booking_id)
        payment = arl.search_payment(payment_code)
        result = customer.refund_booking(booking)
        return{ f"Message : {result}, Booking Id : {booking.get_booking_id()} | Amount left: {payment.get_amount()}" }
    except Exception as e:
        return f"เกิดข้อผิดพลาด: {str(e)}"
```

```
class Transaction:
    def __init__(self):
        self.__rollback_actions = []
    def add_rollback(self, action):
        self.__rollback_actions.append(action)
    def commit(self):
        self.__rollback_actions.clear()
    def rollback(self):
        for action in reversed(self.__rollback_actions):
            action()
```

Prompt: ขอคืนเงินตัวที่ซื้อไป

ดำเนินการคืนเงินตัว T1 ให้เลยครับ!

Used demo integration, used a tool >

คืนเงินเรียบร้อยแล้วครับ! 🎉

สรุปการคืนเงิน:

รายการ	รายละเอียด
📅 Booking	B1 (Phrajomklao → Hualamphong)
💰 ยอดคืน	185 บาท
ยอดคงเหลือในบัตร	1,950 บาท

📄 👍 🗨️ ↺

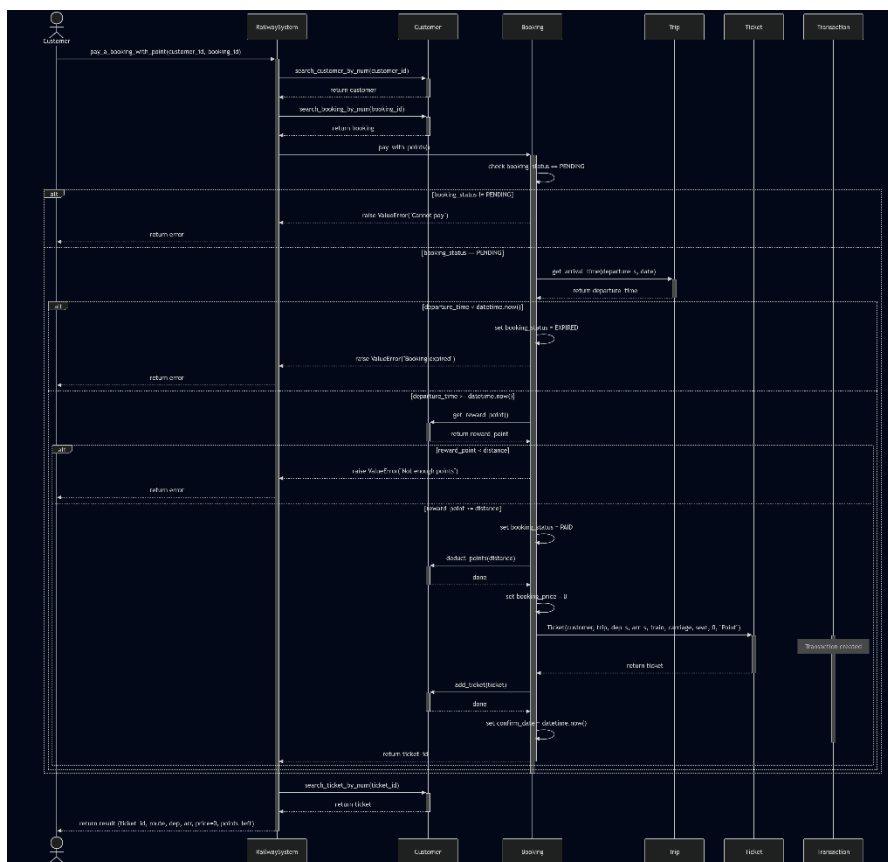
ชื่อ: Pay a Booking with Point

หน้าที่: ใช้โดย Customer เพื่อชำระเงินค่าจองตั๋วรถไฟด้วย Reward Point แทนการชำระด้วยเงิน

พารามิเตอร์: ส่ง customer_id, booking_id

เซอร์วิสที่เกิดขึ้น: เป็นการหัก Reward Point และออกตั๋วโดยระบบจะแสดงผลเป็น ticket_id, route, departure, arrival, seat, price = 0, points_deducted, points_remaining

Sequence diagram:



Actor: Customer

Flow:

1. Customer ส่ง customer_id, booking_id เข้ามาที่ระบบผ่าน pay_a_booking_with_point()
2. ระบบค้นหาข้อมูล Customer ด้วย search_customer_by_num(customer_id) แล้วค้นหา Booking ด้วย search_booking_by_num(booking_id) จาก Customer
3. ระบบเรียก pay_with_points() บน Booking เพื่อเริ่มกระบวนการชำระด้วย Point
4. Booking ตรวจสอบว่า booking_status เป็น PENDING
5. alt [booking_status != PENDING:] หาก Booking มีสถานะไม่ใช่ PENDING ระบบจะ raise ValueError("Cannot pay") และ return error กลับไปยัง Customer

6. alt [booking_status == PENDING:] หาก Booking มีสถานะเป็น PENDING ระบบจะเรียก get_arrival_time(departure_s, date) บน Trip เพื่อตั้งเวลาออกเดินทาง แล้วได้รับ departure_time กลับมา
7. alt [departure_time < datetime.now():] หากเวลาออกเดินทางผ่านมาแล้ว Booking จะ set booking_status = EXPIRED และ raise ValueError("Booking expired") แล้ว return error กลับไปยัง Customer
8. alt [departure_time >= datetime.now():] หากเวลาออกเดินทางยังไม่ผ่าน ระบบจะเรียก get_reward_point() บน Customer แล้วได้รับ reward_point กลับมา
9. alt [reward_point < distance:] หาก Point ไม่เพียงพอ ระบบจะ raise ValueError("Not enough points") และ return error กลับไปยัง Customer
10. alt [reward_point >= distance:] หาก Point เพียงพอ ระบบดำเนินการดังนี้
11. Booking set booking_status = PAID
12. Booking เรียก deduct_points(distance) บน Customer เพื่อหัก Point ตามระยะทาง แล้วได้รับ done กลับมา
13. Booking set booking_price = 0
14. Booking สร้าง Ticket object ด้วย Ticket(customer, trip, dep_s, arr_s, train, carriage, seat, 0, "Point") พร้อมสร้าง Transaction เพื่อรองรับการ rollback แล้วได้รับ ticket กลับมา
15. Booking เรียก add_ticket(ticket) บน Customer เพื่อบันทึก Ticket แล้วได้รับ done กลับมา
16. Booking set confirm_date = datetime.now() และ return ticket_id กลับมา
17. ระบบค้นหา Ticket ด้วย search_ticket_by_num(ticket_id) จาก Customer แล้วได้รับ ticket กลับมา
18. ระบบส่งข้อมูลตอบกลับไปยัง Customer ว่าชำระด้วย Point สำเร็จ พร้อม ticket_id, เส้นทาง, ต้นทาง, ปลายทาง, price=0 และ points_left

Code ที่เกี่ยวข้อง

```
@mcp.tool()
def pay_a_booking_with_point(customer_id: str, booking_id: str):
    """pay booking with point"""
    try:
        customer = arl.search_customer_by_num(customer_id)
        booking = customer.search_booking_by_num(booking_id)

        result = booking.pay_with_points()
        ticket = customer.search_ticket_by_num(result)

        return(f"Message : Booking has been paid",
              f"Created Date : {ticket.get_ticket_create_date().strftime('%d/%m/%Y %H:%M')}",
              f"Ticket Id : {ticket.get_ticket_id()}",
              f"Route : {ticket.get_ticket_route().get_route_name()}",
              f"Departure Station : {ticket.get_ticket_departure().get_station_name()}",
              f"Departure Time : {ticket.get_ticket_trip().get_arrival_time(ticket.get_ticket_departure(), ticket.get_ticket_date().date()).strftime('%d/%m/%Y %H:%M')}",
              f"Arrival Station : {ticket.get_ticket_trip().get_station_name()}",
              f"Arrival Time : {ticket.get_ticket_trip().get_arrival_time(ticket.get_ticket_arrival(), ticket.get_ticket_date().date()).strftime('%d/%m/%Y %H:%M')}",
              f"Price : {ticket.get_ticket_price()}"
              f"Amount of point left : {customer.get_reward_point()}"
              )
    except Exception as e:
        return f"เกิดข้อผิดพลาด: {str(e)}"
```

```
class Booking:
    def pay_with_points(self):
        if self.__booking_status != BookingStatus.PENDING:
            raise ValueError("Cannot pay")
        departure_time = self.__trip.get_arrival_time(
            self.__departure_s, self.__date.date())
        if departure_time < datetime.now():
            self.__booking_status = BookingStatus.EXPIRED
            raise ValueError("Booking expired")
        self.__booking_status = BookingStatus.PAID
        if self.__customer.get_reward_point() < self.__distance:
            raise ValueError("Not enough points")
        self.__customer.deduct_points(self.__distance)
        self.__booking_price = 0
        ticket = Ticket(self.__customer, self.__trip, self.__departure_s,
                        self.__arrival_s, self.__train, self.__carriage,
                        self.__seat, self.__booking_price, "Point")
        self.__customer.add_ticket(ticket)
        self.__confirm_date = datetime.now()
        return ticket.get_ticket_id()
```

```
class Customer(User):
    def collect_points(self, distance):
        self.__reward_point += distance
    def deduct_points(self, distance):
        self.__reward_point -= distance
    def get_reward_point(self):
        return self.__reward_point
```

Prompt:จองตั๋วให้ C1 จาก หัวลำโพงไป อยุธยา รอบแรก ที่นั่งธรรมดา ถ้ามีจ่ายเงินด้วยบัตร 123456 ได้เลย

ต่อไป จองเส้นทางจาก หัวลำโพง ไป พระจอมเกล้า โดยจ่ายเงินเป็น แต้มสะสมคะแนน

ดำเนินการทั้งสองเส้นทางเสร็จเรียบร้อยแล้วครับ!

สรุปการจองทั้งหมด:

	เส้นทางที่ 1	เส้นทางที่ 2
เส้นทาง	Hualamphong → Ayutthaya	Hualamphong → Phrajomklao
วันที่	10/03/2026	10/03/2026
เวลา	09:00 → 09:31	08:45 → 08:48
ที่นั่ง	Ca3 - 1B (Normal)	Ca6 - 1A (Normal)
Ticket	T3	T4
ชำระ	บัตร 123456 / 695 บาท	แต้มสะสม (คงเหลือ 236 แต้ม)